

Q1.1 Reconstruction Loss

Part 1: Gaussian Log-Likelihood $\Leftrightarrow$ MSE

Given the decoder distribution:

$$p_{\theta}(x \mid z) = \mathcal{N}(f_{\theta}(z), \sigma^2 I),$$

the log-likelihood of a single data point $x \in \mathbb{R}^D$ is:

$$\log p_{\theta}(x \mid z) = \log \left[\frac{1}{(2\pi\sigma^2)^{D/2}} \exp \left(-\frac{\|x - f_{\theta}(z)\|^2}{2\sigma^2} \right) \right].$$

Expanding the logarithm:

$$\log p_{\theta}(x \mid z) = -\frac{D}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \|x - f_{\theta}(z)\|^2.$$

Maximising this with respect to θ : the first term is a constant (independent of θ), so maximising $\log p_{\theta}(x \mid z)$ is equivalent to:

$$\max_{\theta} \left[-\frac{1}{2\sigma^2} \|x - f_{\theta}(z)\|^2 \right] \iff \min_{\theta} \|x - f_{\theta}(z)\|^2.$$

Normalising by the number of dimensions D yields the **Mean Squared Error**:

$$\min_{\theta} \underbrace{\frac{1}{D} \|x - f_{\theta}(z)\|^2}_{\text{MSE}(x, f_{\theta}(z))}$$

Hence, under a fixed-variance Gaussian decoder, maximising the log-likelihood is *exactly* equivalent to minimising MSE.

Part 2: Implications of a Deterministic Decoder

When the decoder is implemented deterministically—outputting only the mean $f_{\theta}(z)$ while ignoring σ^2 —the model collapses the full conditional distribution $p_{\theta}(x \mid z)$ to a point estimate. This has several implications for generated image quality:

- **Blurry outputs.** MSE penalises all pixel deviations equally and is minimised by predicting the *average* over all plausible reconstructions. When the posterior $q_{\phi}(z \mid x)$ is diffuse, the decoder averages over multiple modes, producing blurry, low-sharpness images.

- **No per-pixel uncertainty.** Because the variance $\sigma^2 I$ is fixed and discarded at inference time, the model cannot express that some pixels are more uncertain than others. Fine-grained textures and sharp edges—which require modelling high-frequency variation—are therefore systematically suppressed.

In summary, a deterministic MSE decoder trades off sample sharpness and perceptual realism for training stability, which is why VAE-generated images are often noticeably blurrier than those from, e.g., GANs or diffusion models.

Part 3: Bernoulli Decoder and Binary Cross-Entropy

Corresponding loss function. Modelling each pixel independently as a Bernoulli variable gives:

$$p_{\theta}(x | z) = \prod_{i=1}^D \text{Bern}(x_i; \hat{x}_i) = \prod_{i=1}^D \hat{x}_i^{x_i} (1 - \hat{x}_i)^{1-x_i},$$

where $\hat{x}_i = [f_{\theta}(z)]_i \in (0, 1)$ is the decoder’s predicted probability for pixel i .

Taking the negative log-likelihood:

$$-\log p_{\theta}(x | z) = -\sum_{i=1}^D \left[x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i) \right] =: \mathcal{L}_{\text{BCE}}(x, \hat{x}).$$

Maximising the log-likelihood is therefore equivalent to minimising the **Binary Cross-Entropy (BCE)** loss.

Why BCE is more principled for MNIST. MNIST pixels are either black or white, i.e. $x_i \in \{0, 1\}$.

- **Correct support.** Bernoulli random variables live on $\{0, 1\}$, matching the binary nature of MNIST pixels exactly. MSE implicitly assumes continuous, Gaussian-distributed pixels, which is a distributional mismatch.
- **Probabilistic interpretation.** The decoder output $\hat{x}_i$ is interpreted as $P(x_i = 1 | z)$, giving a well-defined generative story: sample each pixel independently from $\text{Bern}(\hat{x}_i)$.
- **Asymmetric penalty.** BCE penalises confident wrong predictions (e.g., $\hat{x}_i \approx 0$ when $x_i = 1$) much more heavily than MSE does, providing stronger gradient signal for near-binary pixels and leading to sharper reconstructions on binarised data.

Therefore, for MNIST (or any dataset with binary pixels), the Bernoulli decoder with BCE loss is a more principled and better-calibrated choice than a Gaussian decoder with MSE loss.

1.2 Closed-Form KL Divergence

We are given:

$$q_\phi(z \mid x) = \mathcal{N}(\mu, \text{diag}(\sigma^2)), \quad p(z) = \mathcal{N}(0, I),$$

where $z \in \mathbb{R}^d$, $\mu = (\mu_1, \dots, \mu_d)^\top$, and $\sigma^2 = (\sigma_1^2, \dots, \sigma_d^2)^\top$.

$$D_{\text{KL}}(q_\phi(z \mid x) \parallel p(z)) = \int_{\mathbb{R}^d} q_\phi(z \mid x) \log \left(\frac{q_\phi(z \mid x)}{p(z)} \right) dz \quad (1)$$

Using the fact that both distributions factorize due to diagonal covariance:

$$\log \left(\frac{\prod_{k=1}^d q_k(z_k)}{\prod_{k=1}^d p_k(z_k)} \right) = \sum_{j=1}^d \log \left(\frac{q_j(z_j)}{p_j(z_j)} \right) \quad (2)$$

Substituting into the integral and applying linearity:

$$D_{\text{KL}} = \sum_{j=1}^d \int_{\mathbb{R}^d} \left(\prod_{k=1}^d q_k(z_k) \right) \log \left(\frac{q_j(z_j)}{p_j(z_j)} \right) dz_1 \dots dz_d \quad (3)$$

Separating the variables:

$$D_{\text{KL}} = \sum_{j=1}^d \left(\int_{\mathbb{R}} q_j(z_j) \log \frac{q_j(z_j)}{p_j(z_j)} dz_j \right) \prod_{k \neq j} \underbrace{\left(\int_{\mathbb{R}} q_k(z_k) dz_k \right)}_{=1} \quad (4)$$

Resulting in:

$$D_{\text{KL}}(q_\phi(z \mid x) \parallel p(z)) = \sum_{j=1}^d D_{\text{KL}}(\mathcal{N}(\mu_j, \sigma_j^2) \parallel \mathcal{N}(0, 1)) \quad (5)$$

It therefore suffices to derive the scalar case for a single dimension j , then sum.

Step 1: KL Divergence from Definition

For continuous distributions the KL divergence is:

$$D_{\text{KL}}(q \parallel p) = \int q(z) \log \frac{q(z)}{p(z)} dz = \mathbb{E}_q[\log q(z) - \log p(z)].$$

For one dimension, write out the log-densities explicitly.

Log of q :

$$\log q(z_j) = -\frac{1}{2} \log(2\pi\sigma_j^2) - \frac{(z_j - \mu_j)^2}{2\sigma_j^2}.$$

Log of p :

$$\log p(z_j) = -\frac{1}{2} \log(2\pi) - \frac{z_j^2}{2}.$$

Therefore:

$$\log q(z_j) - \log p(z_j) = -\frac{1}{2} \log \sigma_j^2 - \frac{(z_j - \mu_j)^2}{2\sigma_j^2} + \frac{z_j^2}{2}.$$

Step 2: Take the Expectation Under q

$$D_{\text{KL}}(\mathcal{N}(\mu_j, \sigma_j^2) \| \mathcal{N}(0, 1)) = \mathbb{E}_{z_j \sim q} \left[-\frac{1}{2} \log \sigma_j^2 - \frac{(z_j - \mu_j)^2}{2\sigma_j^2} + \frac{z_j^2}{2} \right].$$

Evaluate each term using standard Gaussian moments ($\mathbb{E}[z_j] = \mu_j$, $\mathbb{E}[(z_j - \mu_j)^2] = \sigma_j^2$, $\mathbb{E}[z_j^2] = \sigma_j^2 + \mu_j^2$):

$$\mathbb{E} \left[-\frac{1}{2} \log \sigma_j^2 \right] = -\frac{1}{2} \log \sigma_j^2, \quad (6)$$

$$\mathbb{E} \left[-\frac{(z_j - \mu_j)^2}{2\sigma_j^2} \right] = -\frac{\sigma_j^2}{2\sigma_j^2} = -\frac{1}{2}, \quad (7)$$

$$\mathbb{E} \left[\frac{z_j^2}{2} \right] = \frac{\sigma_j^2 + \mu_j^2}{2}. \quad (8)$$

Step 3: Combine the Three Terms

$$D_{\text{KL}}(\mathcal{N}(\mu_j, \sigma_j^2) \| \mathcal{N}(0, 1)) = -\frac{1}{2} \log \sigma_j^2 - \frac{1}{2} + \frac{\sigma_j^2 + \mu_j^2}{2}.$$

Factor out $-\frac{1}{2}$ and rearrange:

$$= \frac{1}{2} (\sigma_j^2 + \mu_j^2 - 1 - \log \sigma_j^2) = -\frac{1}{2} (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

Step 4: Sum Over All d Dimensions

Since all dimensions are independent:

$$D_{\text{KL}}(q_\phi(z | x) \| p(z)) = -\frac{1}{2} \sum_{j=1}^d (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

This completes the derivation.

Q2.1 Reparameterization

Implementation done in code file.

Why the Trick Is Necessary

Training the VAE requires minimising the ELBO loss:

$$\mathcal{L}(\theta, \phi; x) = \underbrace{\mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x|z)]}_{\text{reconstruction}} - D_{\text{KL}}(q_\phi(z|x) \parallel p(z)).$$

To optimise this with gradient descent we need $\nabla_\phi \mathcal{L}$, which requires differentiating *through the sampling step* $z \sim \mathcal{N}(\mu_\phi, \sigma_\phi^2)$. The stochastic node z depends on ϕ through the distribution parameters, but a raw `.sample()` call yields a concrete number with **no gradient attached**—the computational graph is severed at the sampling step.

Instead of sampling z directly, we factor the randomness out:

$$\varepsilon \sim \mathcal{N}(0, I), \quad z = \mu + \varepsilon \odot \sigma.$$

Now z is a *deterministic, differentiable function* of (μ, σ) —both of which are outputs of the encoder network q_ϕ . The stochasticity lives entirely in ε , which does *not* depend on ϕ . Gradients therefore flow back through z to μ and σ , and onward to ϕ :

$$\frac{\partial \mathcal{L}}{\partial \phi} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial (\mu, \sigma)} \cdot \frac{\partial (\mu, \sigma)}{\partial \phi}.$$

All three Jacobians exist and are computed automatically by PyTorch's autograd engine.

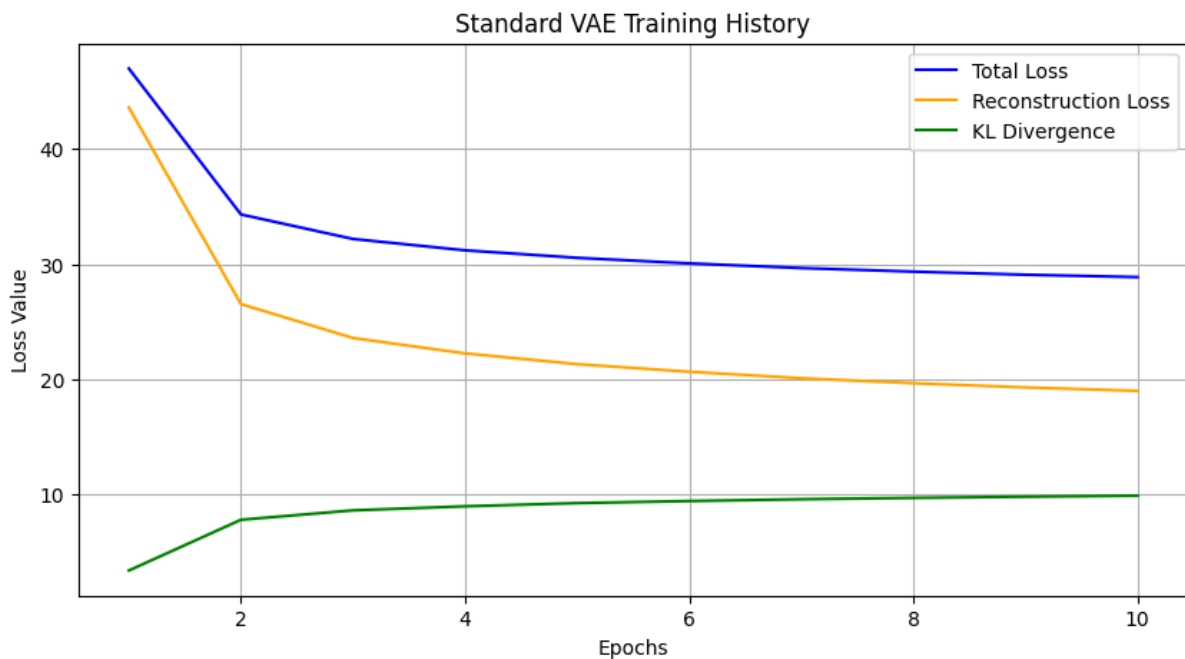
Problem with `torch.distributions.Normal(mu, sigma).sample()` and Backpropagate

1. **Gradient of the reconstruction term w.r.t. encoder parameters is zero.**
`.sample()` draws a value from the distribution but registers no gradient function in the autograd graph. The sampled tensor z has `requires_grad=False`, so $\partial \log p_\theta(x|z) / \partial \phi = 0$ as far as autograd is concerned. The encoder receives *no training signal* from the reconstruction loss.
2. **The encoder only learns from the KL term.** The KL divergence is computed analytically from μ and $\log \sigma^2$ and does have valid gradients w.r.t. ϕ . However, this alone simply pushes the posterior towards the prior $\mathcal{N}(0, I)$, giving no incentive to encode useful information about x .
3. **The model degenerates.** Without a reconstruction gradient reaching the encoder, the encoder cannot learn a meaningful latent representation. The decoder also cannot be jointly optimised with the encoder, so the generative model fails to reconstruct inputs faithfully.

Q2.2 Loss Function

Implemented in code file.

Q2.3 Training Loop

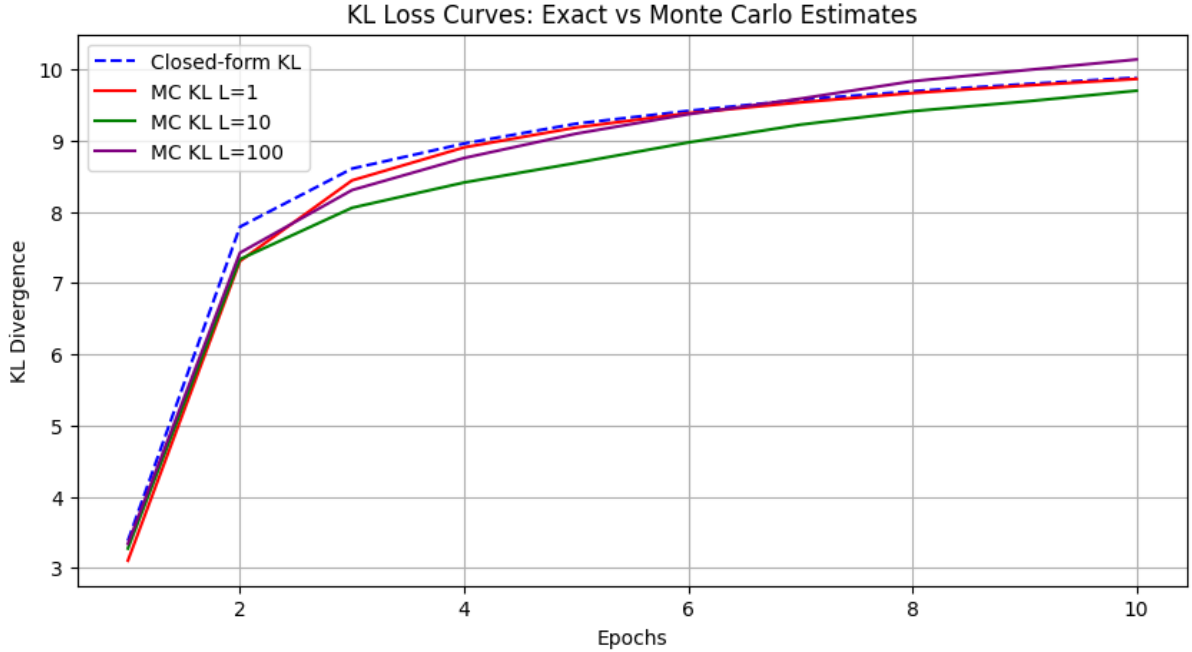


Observed training dynamics (from the plot).

- **Reconstruction loss** (orange) dominates early training and falls steeply in the first 2 epochs as the decoder learns to reproduce input structure.
- **KL divergence** (green) starts near zero (the posterior has not yet diverged from the prior) and rises gradually as the encoder learns informative latent representations.
- **Total loss** (blue) is the sum of both and converges smoothly, indicating stable training without posterior collapse.

The reparameterization trick is therefore not merely a convenience—it is what makes end-to-end gradient-based training of VAEs possible.

Q2.4 Monte Carlo KL Approximation



Implementation done in code file.

2.4.1 Qualitative Comparison of $L = 1$ vs. $L = 100$

In the provided plot, the KL divergence curves for $L = 1$ and $L = 100$ appear remarkably similar and follow the same smooth trajectory as the analytical closed-form solution.

- **Theoretical Expectation:** From a statistical standpoint, an estimator with a sample size of $L = 1$ should exhibit significant variance compared to $L = 100$. One would expect the $L = 1$ curve to be jagged and noisy, reflecting the high variance of single-point stochastic gradients.
- **Empirical Observation:** The observed smoothness in the plot is a result of **epoch-level averaging**. While the gradient update for a single data point is noisy when $L = 1$, the plotted values represent the mean loss over the entire dataset across an epoch. This aggregation serves as a powerful smoothing filter, effectively masking the batch-level variance and making the stochastic estimate converge toward the expected value at the macro level.

2.4.2 The Impact of High Variance in Early Training

High variance in gradient estimates is particularly detrimental during the initial stages of the optimization process for several reasons:

1. **Optimization Instability:** Early in training, the network parameters ϕ are randomly initialized, leading to volatile predicted values for μ and $\log \sigma^2$. If the KL gradient estimate is also highly noisy, the resulting weight updates can be erratic, potentially pushing the model into regimes of numerical instability, such as the underflow or overflow of standard deviation values.

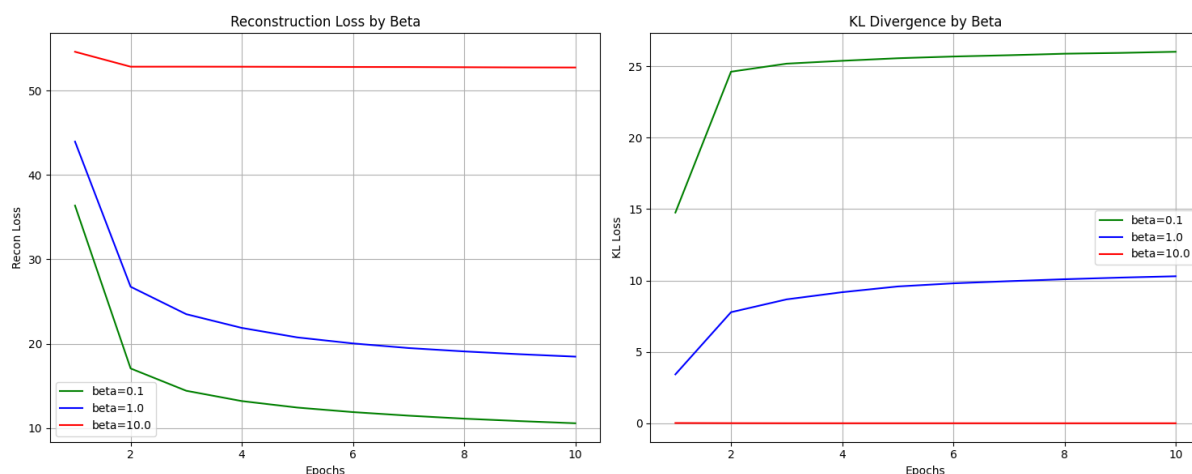
2. **Signal-to-Noise Ratio:** The learning signal from the reconstruction term is delicate at the start. Excessive noise from the KL term can "wash out" this signal, causing the optimizer to move in directions that do not actually improve the model's representative capacity, leading to slow convergence or suboptimal local minima.
3. **Premature Posterior Collapse:** A noisy KL signal may inadvertently drive the encoder to simply output the prior (making $\mu \approx 0$ and $\sigma \approx 1$) as a "safe" path to minimize the total loss, before the decoder has had a chance to learn how to use the latent space meaningfully.

2.3 Advantages of the Closed-Form Solution

Even when computational resources allow for a large number of samples L , the closed-form expression is fundamentally superior for two reasons:

- **Exact Gradient Signal (Zero Variance):** The closed-form solution provides the exact value of the KL divergence. By eliminating sampling noise entirely, it provides a perfectly clean gradient, allowing the optimizer to take precise steps in parameter space without the interference of Monte Carlo variance.
- **Computational Parsimony:** Calculating the closed-form requires only basic arithmetic operations—specifically, $O(d)$ time and memory complexity for d latent dimensions. In contrast, the Monte Carlo approach requires L calls to a random number generator, the reparameterization of L samples, and $2L$ log-probability evaluations. The closed-form is faster, consumes less memory, and is mathematically optimal.

Q3.1 Training

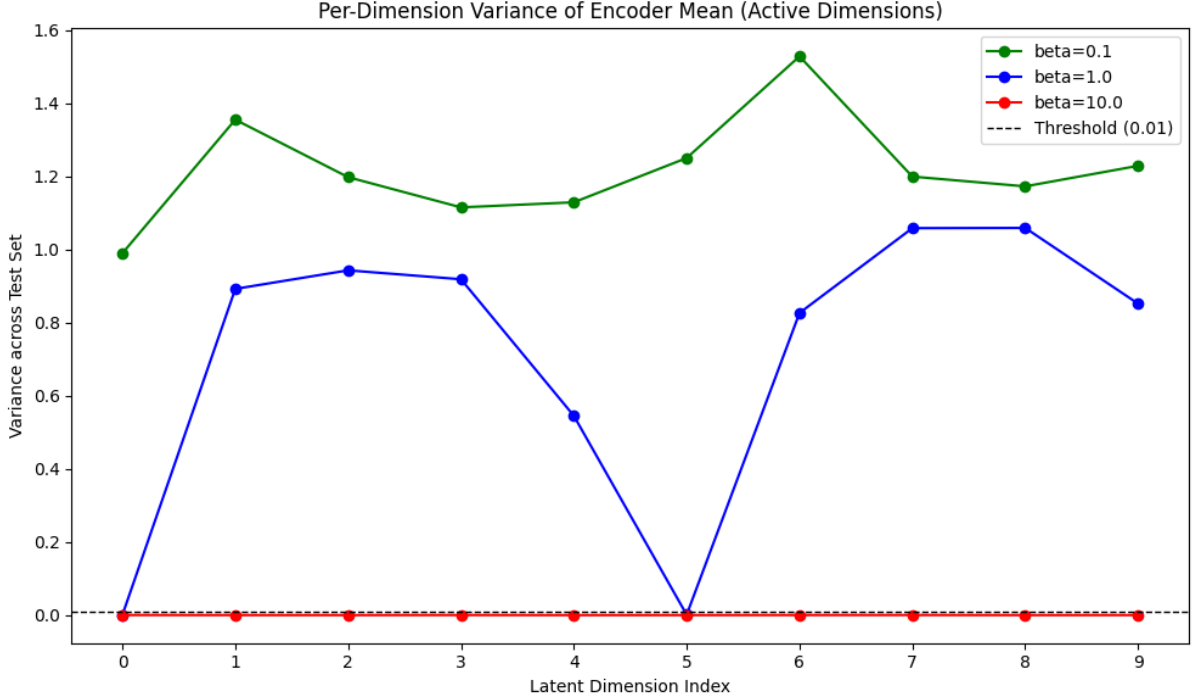


Observations

The training curves demonstrate the fundamental trade-off between reconstruction accuracy and latent regularization:

- **Low β (0.1):** Prioritizes the reconstruction term. The model achieves the lowest reconstruction loss (≈ 10) at the cost of a high KL divergence (≈ 26), indicating a latent space that is highly specialized to the training data but potentially less structured.
- **Standard β (1.0):** Represents the standard VAE, providing a balance between the two terms.
- **High β (10.0):** Leads to **Posterior Collapse**. The KL divergence vanishes to near zero almost immediately. Because the penalty for deviating from the prior is too high, the encoder fails to pass useful information through the latent bottleneck, causing the reconstruction loss to plateau at a very high value (≈ 53).

Q3.2 Posterior Collapse Analysis



(This plot changes significantly for $\beta = 0.1, 1.0$ with each run)

Observations for $\beta = 10.0$

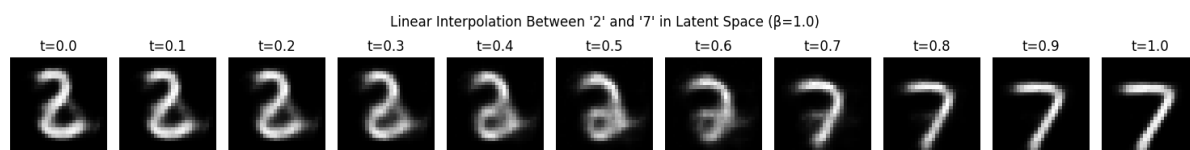
Under the setting of $\beta = 10.0$, the plot of per-dimension variance shows that all latent dimensions fall below the threshold of 0.01. The variance of the encoder means across the test set is effectively zero, indicating that the encoder maps every input x to the same region of the latent space. This is the definitive signature of **Posterior Collapse**.

Explanation

Incentive to Ignore Input: The KL divergence term $D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))$ measures the information "cost" of the latent bottleneck. When $\beta \gg 1$, the optimization objective heavily penalizes any information flow from x to z . The model minimizes the total loss by setting $q_\phi(\mathbf{z}|\mathbf{x}) \approx p(\mathbf{z})$, thereby reducing the KL term to zero. In this regime, the regularization pressure outweighs the reconstruction incentive.

Lack of Signal to the Decoder: When the encoder ignores the input, the latent variable $\mathbf{z}$ becomes independent of $\mathbf{x}$ ($\mathbf{z} \sim \mathcal{N}(0, I)$). From the decoder's perspective, $\mathbf{z}$ is merely pure noise. Since there is no mutual information between the input $\mathbf{x}$ and the code $\mathbf{z}$, the decoder cannot utilize $\mathbf{z}$ to reconstruct specific features. Instead, the decoder ignores the latent signal and defaults to generating the mean of the data distribution to minimize the reconstruction error.

Q4.1 Linear Interpolation



We analyze the latent space geometry by interpolating between the mean vectors of a digit "2" (μ_A) and a digit "7" (μ_B):

$$z_t = (1 - t)\mu_A + t\mu_B, \quad t \in \{0.0, 0.1, \dots, 1.0\} \quad (9)$$

Transition Analysis

The resulting images show a **smooth semantic transition**. Rather than a pixel-wise overlay, the digits undergo a structural transformation. The bottom loop of the "2" gradually retracts and the diagonal bar re-orient until the image becomes a "7". This confirms that the model has learned a continuous latent manifold where intermediate points correspond to plausible, digit-like structures.

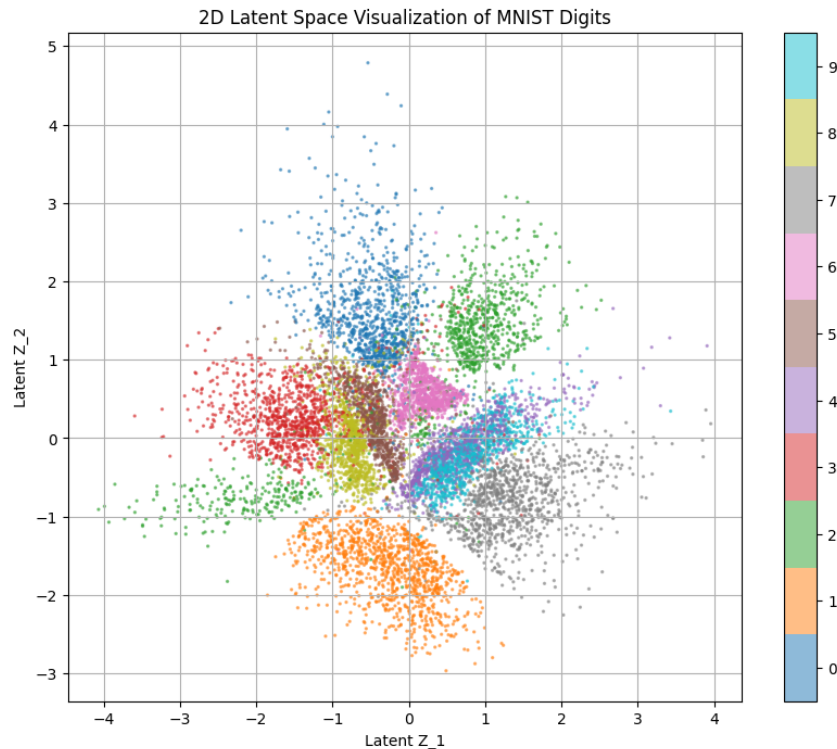
The Role of KL Regularization

The KL term is fundamental to this behavior for two reasons:

1. **Manifold Continuity:** By penalizing the divergence from a standard Gaussian prior $\mathcal{N}(0, I)$, the KL term prevents the latent space from becoming fragmented. It forces the encoder to map similar inputs to nearby regions, ensuring that the space between "2" and "7" is populated with meaningful intermediate representations.
2. **Local Robustness:** Unlike a standard Autoencoder, which maps inputs to discrete points, a VAE maps inputs to distributions. The decoder must learn to reconstruct the digit from any sample in that distribution's neighborhood. This overlap between distributions ensures that the transitions between classes are gradual rather than abrupt.

In a standard Autoencoder, the latent space is unconstrained, often leading to "holes" where interpolations result in non-semantic noise because the decoder was never trained on those unmapped regions.

Q4.2 2-D Latent Space Visualization



1. Separation of Digit Classes

- The 2D latent space demonstrates a clear global organization where most digit classes occupy distinct, identifiable regions of the coordinate plane.
- Highly distinctive digits, such as '0' (located at the top) and '1' (located at the bottom), are the most isolated groups, reflecting their unique topological structures.
- However, there is a central region where multiple clusters converge, leading to significant overlap at the boundaries of classes.
- This overlap is expected in a $Z = 2$ bottleneck, as the model struggles to perfectly disentangle all ten classes within such a low-dimensional constraint.

2. Clustering of Visually Similar Digits

- The model successfully captures the semantic topology of the dataset by placing visually similar digits in adjacent or overlapping latent neighborhoods.
- Digits characterized by sharp angles and straight strokes, specifically '4', '7', and '9', form a contiguous cluster in the lower-right quadrant.
- Conversely, digits with rounded or curved components, such as '3', '5', and '8', are grouped closely together in the left-center portion of the plot.
- This clustering confirms that the VAE has learned a smooth manifold where geometric proximity in the latent space corresponds to structural similarity in the original pixel space.